```python
# -*- coding: utf-8 -*-
"""
Created on Wed Jan  4 12:13:57 2023

@author: mathe
"""

# Import necessary libraries
import tensorflow as tf
import numpy as np
import pandas as pd
from sklearn.preprocessing import MinMaxScaler
from sklearn.model_selection import train_test_split


# Load data from files
xyz_data = np.array(pd.read_csv('Ux.txt', delimiter = "\t"))[:,1:4]
ux = np.array(pd.read_csv('Ux.txt', delimiter = "\t"))[:,4]
uy = np.array(pd.read_csv('Uy.txt', delimiter = "\t"))[:,4]
uz = np.zeros((uy.shape[0],1))
displacement = np.concatenate((ux[:,np.newaxis], uy[:,np.newaxis], uz), axis=1)
sxx = np.array( pd.read_csv('Sxx.txt', delimiter = "\t"))[:,4]
syy = np.array( pd.read_csv('Syy.txt', delimiter = "\t"))[:,4]
sxy = np.array( pd.read_csv('Sxy.txt', delimiter = "\t"))[:,4]
col_data = np.array(pd.read_csv('Collocation Points.txt', delimiter = "\t"))[:,1:3]

bc_data = np.array(pd.read_csv('Ux.txt', delimiter = "\t"))
bc_data = np.concatenate((bc_data, uy[:,np.newaxis]), axis=1)

# Create a boolean index that is True for rows where the value in the 3rd column is 10
index = (bc_data[:, 2] == 10)

#BCs is a 3 column array (x,y,z) vectors of displacement BCs or could just be a one column?
boundary_conditions = np.zeros((bc_data.shape[0],1))

# Set the values in to 10 when y = 10
boundary_conditions[index] = 10

# Create a boolean index that is True for rows where the value in the 3rd column is 0 or 10
index = (bc_data[:, 2] == 0) | (bc_data[:, 2] == 10)

# Create a new column with all values set to 0
bc_present = np.zeros((bc_data.shape[0], 1))

# Set the values in the new column to 1 for rows where the index is True
bc_present[index] = 1

## Concatenate the old array and the new column to create the new array
boundary_conditions = np.concatenate((bc_present, boundary_conditions), axis=1)

# Train Test Split
train_points, test_points, train_boundary_conditions, test_boundary_conditions =
train_test_split(points, boundary_conditions, test_size=0.2)
train_points, val_points, train_boundary_conditions, val_boundary_conditions =
train_test_split(train_points, train_boundary_conditions, test_size=0.2)

# Normalize the data
scaler = MinMaxScaler()
points = scaler.fit_transform(train_points)
boundary_conditions = scaler.fit_transform(train_boundary_conditions)
```

```
# Define the inputs to the neural network
points = tf.compat.v1.placeholder(tf.float32, shape=(None, num_points, 3))  # 3D coordinates of the points
boundary_conditions = tf.compat.v1.placeholder(tf.float32, shape=(None, num_points, num_boundary_conditions))  # Boundary conditions at each point

# Concatenate the inputs
inputs = tf.concat([points, boundary_conditions], axis=-1)

# Define the shared MLP and the convolutional layers for feature extraction
model = tf.keras.Sequential()
model.add(tf.keras.layers.Conv1D(64, kernel_size=1, activation='relu', input_shape=(num_points, 3+num_boundary_conditions)))
model.add(tf.keras.layers.Conv1D(64, kernel_size=1, activation='relu'))
model.add(tf.keras.layers.Conv1D(64, kernel_size=1, activation='relu'))

# Define the max pooling layer
model.add(tf.keras.layers.GlobalMaxPool1D())

# Define the fully connected layers
model.add(tf.keras.layers.Dense(512, activation='relu'))
model.add(tf.keras.layers.Dense(256, activation='relu'))
model.add(tf.keras.layers.Dense(3, activation='linear'))

# Define the loss function
def neo_hookean_loss(displacement, predicted_displacement):
    # Define the neo-hookean energy density
    I1 = tf.reduce_sum(tf.square(predicted_displacement), axis=-1)
    J = tf.sqrt(tf.reduce_prod(I1))
    W = (mu/2)*(I1 - 3) - mu*tf.log(J)

    # Define the loss as the mean squared error between the predicted displacement and the true displacement
    loss = tf.reduce_mean(tf.square(predicted_displacement - displacement)) + tf.reduce_mean(W)
    return loss

# Define the optimizer
optimizer = tf.train.AdamOptimizer()

# Define the training operation
train_op = optimizer.minimize(neo_hookean_loss)

# Train the model
with tf.Session() as sess:
    sess.run(tf.global_variables_initializer())
    for step in range(num_steps):
        _, loss_value = sess.run([train_op, neo_hookean_loss], feed_dict={points: xyz_data, boundary_conditions: boundary_conditions, displacement: displacement})
        if step % 100 == 0:
            print("Step: {}, Loss: {}".format(step, loss_value))
```